15/09/2020

# Experiment - 7 : SLIDING WINDOW PROTOCOL

Aim: Write a program to implement flow control at data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another

Program should achieve atleast below given requirements. You can make it a bidirectional program wherein receiver is sending its data frames with acknowledgement (Piggybacking).

Create a sender program with the following features :-

(i) Input Window size from the user.

(ii) Input a text message from the user.

(iii) Consider 1 character per frame.

(iv) Create a frame with the following fields [Frame no., DATA].

(v) Send the frames (Print the output on screen & save it in a file called Sender_Buffer)

(vi) Wait for the acknowledgement from the Receiver (Induce delay in the pgm).

(vii) Read a file called Receiver_Buffer.

(viii) Check ACK field for the Acknowledgement number.

(ix) If the Acknowledgement number is as expected, send new set of frames accordingly, (overwrite the Sender_Buffer file with new frames) Else if NACK is received, resend the frames accordingly. (Overwrite the Sender_Buffer with old frame).

Create a receiver file with following features

(i) Reader a file called Sender_Buffer.

(ii) Check the frame no.

(iii) If the Frame no. are as expected, write the appropriate ACK no. in the Receiver_Buffer file. Else write NACK no. in the Receiver_Buffer file.

NOTE: ~~It~~ Induce error & verify the behaviour of the program. Manually change the Frame no. & Ack no. in the files).

Observation :-

Sender.py :-

```
import os
import time

BASE = os.path.dirname(os.path.abspath(__file__))
SENDER_BUFFER = os.path.join(BASE, "Sender_Buffer.txt")
RECEIVER_BUFFER = os.path.join(BASE, "Receiver_Buffer.txt")
POLL_DELAY = 1


def atomic_write(path, text):
    """Write to a temp file, then rename,
       retrying if Windows has the file locked."""
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        f.write(text)
    for _ in range(30):
        try:
            os.replace(tmp, path)
            return
        except PermissionError:
            time.sleep(0.1)
    raise PermissionError(f"Could not write {path}")
```

```
def safe_read(path):
    try:
        with open(path) as f:
            return f.read().strip()
    except (FileNotFoundError, PermissionError):
        return ""

def main():
    window = int(input("Enter window size: "))
    message = input("Enter text message: ")
    n = len(message)

    print("Writing: ", SENDER_BUFFER)
    print("Reading: ", RECEIVER_BUFFER)

    base = 0
    tx = 0
    piggy_received = ""

    atomic_write(RECEIVER_BUFFER, "")

    while base < n:
        tx += 1
        end = min(base + window, n)
        lines = [f"TX, {tx}"] + [f"{i},
            {message[i]}" for i in range(base, end)]
```

```
print(f"\n--- Transmission {tx}:
      sending frames {base} to {end-1} ---")
for i in range(base, end):
    print(f"  Frame [{i}] | {message[i]!r}")

atomic_write(RECEIVER_BUFFER, "\n")
atomic_write(SENDER_BUFFER, "\n".join(lines)
                            + "\n")

print("Waiting for acknowledgement...")
while True:
    time.sleep(POLL_DELAY)
    reply = safe_read(RECEIVER_BUFFER)
    if reply:
        break

parts = reply.split(",", 2)
try:
    kind, num = parts[0].strip().upper(),
                int(parts[1])
except (IndexError, ValueError):
    print(f"Corrupted reply '{reply}'
          → resending same window")
    continue

if len(parts) >= 2 and parts[2].strip():
    piggy_received += parts[2].strip()
    print(f"Piggybacked data from receiver:
          {parts[2].strip()!r}")
```

```
if kind == "ACK" and base <= num <= end:
    print(f"ACK {num} received → window slides to {num}")
    base = num

elif kind == "NACK" and base <= num < end:
    print(f"NACK {num} received → resending from frame {num}")
    base = num

else:
    printf(f"Unexpected reply '{reply}' → resending same window")

tx += 1
atomic_write(SENDER_BUFFER, f"TX,{tx}\nEOF\n")
print("\nAll frames delivered successfully.")
print("Data piggybacked from receiver:",
      piggy_received)

if __name__ == "__main__":
    main()
```

receiver.py

```
import os
import time

BASE = os.path.dirname(os.path.abspath(__file__))
SENDER_BUFFER = os.path.join(BASE, "Sender_Buffer.txt")
RECEIVER_BUFFER = os.path.join(BASE, "Receiver_Buffer.txt")

POLL_DELAY = 1

PROCESS_DELAY = 8

def atomic_write(path, text):
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        f.write(text)
    for _ in range(30):
        try:
            os.replace(tmp, path)
            return
        except PermissionError:
            time.sleep(0.1)

    raise PermissionError(f"Could not write {path}")

def read_sender_buffer():
    try:
        with open(SENDER_BUFFER) as f:
            return f.read().splitlines()
    except
```

```
def remove_if_exists(path):
    try:
        os.remove(path)
    except OSError:
        pass

def main():
    reply_msg = input("Enter receiver's own
                message (piggybacked with ACKs): ")

    remove_if_exists(SENDER_BUFFER)
    remove_if_exists(RECEIVER_BUFFER)
    remove_if_exists(SENDER_BUFFER + ".tmp")
    remove_if_exists(RECEIVER_BUFFER + ".tmp")

    expected = 0
    last_txt = None
    reply_pos = 0
    received = ""

    print("Receiver waiting for frames...")
    print("Watching:", SENDER_BUFFER)
```

```
while True :
    time.sleep (POLL_DELAY)
    lines = read_sender_buffer()
    if not lines or not lines[0].startswith("TX,"):
        continue
    tx = lines[0]
    if tx == last_tx :
        continue
    last_tx = tx

    print (f"\n--- New transmission detected
                                    ({tx}) ---")
    print (f"Edit Sender_Buffer.txt now to
            induce an error... ({PROCESS_DELAY}s)")
    time.sleep (PROCESS_DELAY)

    lines = read_sender_buffer()
    if not lines :
        continue
    if len(lines) > 1 and lines[1].strip() == "EOF":
        print ("\nSender finished. Message received: ",
                                        received)
        break
```

```
ok = true
for line in lines[1:]:
    line = line.strip()
    if not line:
        continue

    try:
        no, data = line.split(",", 1)
        no = int(no)
    except ValueError:
        print(f"Malformed frame '{line}'")
        ok = false
        break

    if no == expected:
        print(f"Frame {no} [{data!r}] accepted")
        received += data
        expected += 1
    elif no < expected:
        print(f"Frame {no} is a duplicate, ignored")
    else:
        print(f"Frame {no} received, expected {expected} → error")
        ok = false
        break
```

```
piggy = reply_msg[reply_pos : reply_pos + 1]
if ok :
    reply = f"ACK, {expected}, {piggy}"
    reply_pos += 1
else :
    reply = f"NACK, {expected},"
print(f"Writing to Receiver_Buffer : {reply}")
atomic_write(RECEIVER_BUFFER, reply + "\n")

if __name__ == "__main__":
    main()
```

Output :-

SENDER

Frame No : 0 DATA : A

Frame No : 1 DATA : B

Frame No : 2 DATA : C

RECEIVER

Received Frame : 0 DATA : A

Received Frame : 5 DATA : B

Wrong frame received

Receiver Buffer :

('NACK', 1)

SENDER - CHECKING RESPONSE

NACK received : 1

Resending from Frame 1